QUIZMASTER PRO: A JAVA SWING DESKTOP QUIZ APPLICATION

A PROJECT REPORT

Subject: Project Based Learning in Java (24CSP-301 / 24ITP-301)

Submitted by

MOHAMMAD TAHA (24BCS12733)
SARWAR ADHIKARI (24BCS80370)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Chandigarh University

Oct-2026

BONAFIDE CERTIFICATE

Certified that this project report **"QUIZMASTER PRO: A JAVA SWING DESKTOP QUIZ APPLICATION"** is the bonafide work of **"MOHAMMAD TAHA and SARWAR ADHIKARI"** who carried out the project work under my/our supervision.

SIGNATURE

Name of the Head of the Department

HEAD OF THE DEPARTMENT

Department of Computer Science and Engineering

SIGNATURE

Mr. Shariq Rashid Bhat

SUPERVISOR

Academic designation, e.g. Assistant Professor

Department of Computer Science and Engineering

Chandigarh University, NH-05, Chandigarh-Ludhiana Highway, Mohali, Punjab - 140413

Submitted for the project viva-voce examination held on __ / __ / ____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to everyone who supported us in completing this project.

We are deeply thankful to our faculty guide and supervisor, **Mr. Shariq Rashid Bhat**, for his valuable guidance, constructive feedback and constant encouragement throughout the Project Based Learning in Java course. His suggestions helped us to improve the design, the testing and the quality of this report.

We thank the **Head of the Department** and the faculty of the Department of Computer Science and Engineering, **Chandigarh University**, for providing the laboratory facilities and a learning environment in which we could carry out this work.

We are also grateful to our classmates for testing the application and giving honest feedback on its usability, and to our families for their patience and support.

Finally, we acknowledge the open documentation and tutorials of the Java platform, which made it possible to build the whole application with the standard library alone.

MOHAMMAD TAHA (24BCS12733)
SARWAR ADHIKARI (24BCS80370)

TABLE OF CONTENTS

Acknowledgement.....i
List of Figures.....iv
List of Tables.....v
Abstract.....vi
Graphical Abstract.....vii
Abbreviations.....viii
Symbols.....ix
Chapter 1. Introduction.....1
1.1. Client Identification / Need Identification / Identification of Relevant Contemporary Issue.....1
1.2. Identification of Problem.....1
1.3. Identification of Tasks.....2
1.4. Timeline.....3
1.5. Organization of the Report.....3
Chapter 2. Design Flow / Process.....4
2.1. Evaluation and Selection of Specifications / Features.....4
2.2. Design Constraints.....4
2.3. Analysis and Feature Finalization Subject to Constraints.....5
2.4. Design Flow.....6
2.4.1. Design A - Console application with text files.....6
2.4.2. Design B - Swing desktop GUI with JSON files.....6
2.4.3. Design C - JavaFX GUI with a database.....7
2.5. Design Selection.....7
2.6. Implementation Plan / Methodology.....7
2.6.1. Architecture.....7
2.6.2. Flowchart of a quiz session.....8
2.6.3. Class design.....9
2.6.4. Key algorithms.....10
2.6.5. Data design.....11
Chapter 3. Results Analysis and Validation.....12
3.1. Implementation of Solution.....12
3.2. Screens of the Implemented Application.....12

3.2.1. Login and registration....12
3.2.2. Dashboard....13
3.2.3. Category selection....14
3.2.4. Quiz screen....14
3.2.5. Result and review....16
3.2.6. History and settings....17
3.3. Testing, Characterization and Data Validation....18
3.3.1. Automated logic tests....18
3.3.2. End-to-end navigation test....19
3.3.3. Visual and clean-clone validation....20
3.4. Defects Found and Corrected....20
3.5. Interpretation of Results....22
Chapter 4. Conclusion and Future Work....23
4.1. Conclusion....23
4.2. Future Work....23
References....25
Appendix....26
A1. Appendix-1: Project Structure and Data Formats....26
A2. Appendix-2: Selected Source Code....27
A3. Appendix-3: Complete List of Automated Logic Tests....27
A4. Appendix-4: Source Code Repository....28
User Manual....29
M1. Requirements....29
M2. Download, Build and Run (Windows)....29
M3. Run on Linux or macOS....29
M4. Signing In....29
M5. Dashboard and Navigation....30
M6. Taking a Quiz....30
M7. Keyboard Shortcuts....32
M8. Results and Review....32
M9. History and Settings....32
M10. Troubleshooting....33
Achievements....34

LIST OF FIGURES

Figure 1: Graphical abstract of QuizMaster Pro.....vii

Figure 2: Block diagrams of the three alternative designs.....6

Figure 3: Layered architecture of QuizMaster Pro.....8

Figure 4: Flowchart of a quiz session.....9

Figure 5: Simplified class diagram.....10

Figure 6: Login screen.....13

Figure 7: Create-account screen with validation.....13

Figure 8: Dashboard with statistics and recent activity.....14

Figure 9: Category and question-count selection.....14

Figure 10: Quiz screen with timer and four options.....15

Figure 11: After using the 50:50 lifeline and the hint.....15

Figure 12: Answer feedback: correct option in green, wrong choice in red, with explanation.....16

Figure 13: Result screen with score ring, breakdown and grade.....16

Figure 14: Question-by-question review dialog.....17

Figure 15: Quiz history with score, grade and duration.....17

Figure 16: Settings: display name, sound, timer length, data reset.....18

Figure 17: User manual: login screen.....30

Figure 18: User manual: dashboard.....30

Figure 19: User manual: choosing a category and number of questions.....31

Figure 20: User manual: answering a question.....31

Figure 21: User manual: result screen.....32

Figure 22: User manual: settings screen.....33

LIST OF TABLES

Table 1: Project timeline (Gantt chart; columns are week numbers).....3
Table 2: Candidate features and their selection.....4
Table 3: Design constraints considered.....5
Table 4: Weighted comparison of the alternative designs.....7
Table 5: Source packages of the project.....8
Table 6: Tools and technologies used.....12
Table 7: Summary of automated logic tests.....19
Table 8: End-to-end checks on the real application window.....19
Table 9: Defects found during testing and their corrections.....20
Table 10: Automated logic tests and results.....27
Table 11: Keyboard shortcuts during a quiz.....32
Table 12: Common problems and solutions.....33

ABSTRACT

QuizMaster Pro is a desktop quiz application developed in Java with the Swing toolkit and Java 2D graphics to help students practise core Java topics. The project addresses the lack of a free, offline, distraction-free tool that gives instant feedback, tracks progress over time and works without an internet connection or a database server. The application provides secure sign-in with salted SHA-256 password hashing, four topic banks containing sixty multiple-choice questions, a per-question countdown timer, 50:50 and hint lifelines, instant explanations, a score screen with grade and answer review, a history of previous attempts and day-streak statistics. A layered, MVC-style architecture separates the view, model and service packages, and all data are stored in human-readable JSON files handled by a custom parser, so no external library is needed. Three alternative designs were compared and the Swing-plus-JSON design was selected. Testing combined thirty-seven automated logic checks, end-to-end navigation tests and visual inspection of every screen; all checks passed and twenty-three interface and behaviour defects found during review were corrected. The result is a portable, zero-dependency application of 33 classes and about 6,300 lines of code that runs on any computer with JDK 17 or later.

GRAPHICAL ABSTRACT

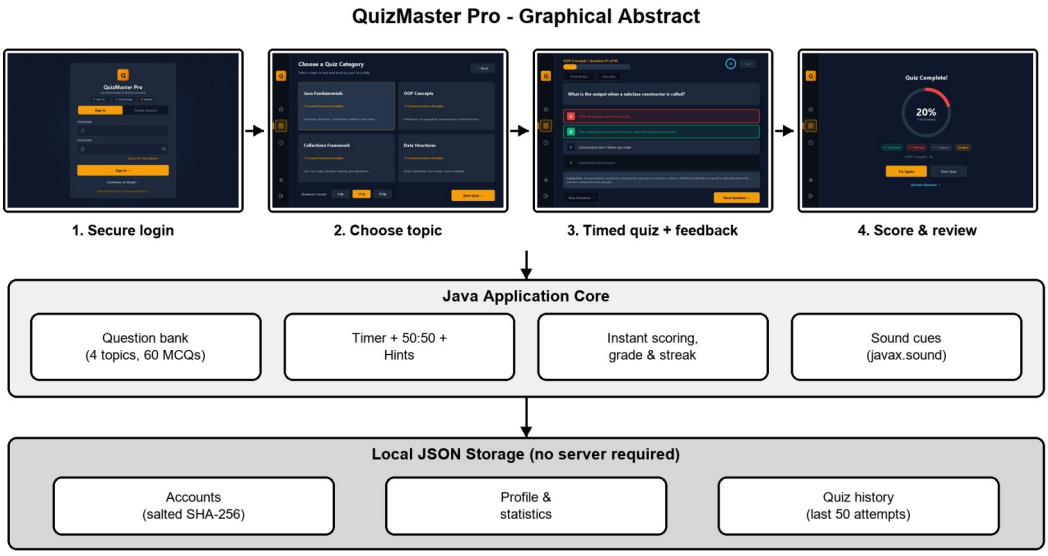


Figure 1: Graphical abstract of QuizMaster Pro

The student signs in, chooses a topic, answers timed questions with instant feedback and sees a graded result. The Java application core applies the question bank, timer, lifelines and scoring, and every account, statistic and attempt is stored in local JSON files.

ABBREVIATIONS

Abbreviation	Full form
API	Application Programming Interface
AWT	Abstract Window Toolkit
CSE	Computer Science and Engineering
CSS	Cascading Style Sheets
DPI	Dots Per Inch
EDT	Event Dispatch Thread (Swing)
FXML	JavaFX Markup Language
GUI	Graphical User Interface
HTML	HyperText Markup Language
IDE	Integrated Development Environment
JAR	Java Archive
JDK	Java Development Kit
JSON	JavaScript Object Notation
MCQ	Multiple-Choice Question
MVC	Model-View-Controller
OOP	Object-Oriented Programming
SHA-256	Secure Hash Algorithm, 256-bit
UI / UX	User Interface / User Experience
UID	Unique Identification Number
WCAG	Web Content Accessibility Guidelines

SYMBOLS

Symbol	Meaning
C	Number of correctly answered questions
N	Total number of questions in a quiz
S	Score in per cent, S = floor(C x 100 / N)
s	Seconds (unit of time)
T	Timer length per question in seconds (15, 30, 45 or 60)
n	Number of quizzes completed by a user
avg	Rolling average score in per cent
%	Per cent

CHAPTER 1.

INTRODUCTION

1.1. Client Identification / Need Identification / Identification of Relevant Contemporary Issue

Learning to program is a skill that is built through repeated, active practice. Reading a textbook or watching a lecture on Java helps a student to understand a concept, but the knowledge only becomes dependable when the student is asked to recall it, apply it to a short piece of code and receive immediate feedback on whether the answer was right. Multiple-choice quizzes are one of the simplest and most widely used ways of providing this kind of practice. They are quick to attempt, they can be scored automatically and they allow a large part of a syllabus to be revised in a short time.

The **client** for this project is the typical undergraduate computer-science student who is learning Java in the Project Based Learning course, together with the faculty who guide such a course and would like their students to revise core topics regularly. The **need** was identified from the way students currently prepare for lab evaluations and viva examinations: they search for scattered question lists, use online quiz websites that require an internet connection, an account and often show advertisements, or simply re-read notes without any way of checking what they have actually retained.

The **contemporary issue** is the rapid shift towards self-paced, digital learning. Students increasingly expect immediate feedback, visible progress and a user experience comparable to the mobile and web applications they use every day. At the same time, many campus laboratories and hostel networks have limited or unreliable connectivity, and handing personal study data to third-party websites raises privacy concerns. A lightweight application that runs entirely on the student's own computer, stores its data locally and still offers a modern interface addresses all three concerns. Optional: add one or two cited statistics on e-learning / online assessment adoption here, if your faculty expects numerical evidence.

1.2. Identification of Problem

Students of Java do not have a free, offline and distraction-free way of practising the core topics of the language and tracking how their understanding improves over time. Existing options are either general-purpose quiz platforms that are not focused on Java, tools that

depend on a constant internet connection and a registered online account, or static question lists that provide no scoring, no explanation of the correct answer and no record of past attempts. As a result, learners cannot easily identify which topics are weak, cannot measure improvement from one attempt to the next, and lose motivation because there is no feedback loop.

1.3. Identification of Tasks

To resolve the problem, the work was divided into the following tasks, which also form the framework of this report:

1. **Requirement analysis** - study similar quiz tools, list the features students need and decide the scope for a one-semester project.
2. **Design** - choose the technology, define the software architecture, the data formats and the visual design language of the application.
3. **Implementation of the data layer** - question banks, user accounts with secure password storage, profile statistics and quiz history.
4. **Implementation of the quiz engine** - question selection and shuffling, per-question timer, scoring, lifelines (50:50 and hint) and result computation.
5. **Implementation of the user interface** - login, dashboard, category selection, quiz, result, history, settings and answer-review screens.
6. **Testing and validation** - automated logic tests, end-to-end navigation tests and visual inspection of every screen; defect fixing.
7. **Documentation** - this report, a user manual and the project repository.

1.4. Timeline

The project was planned over eleven weeks. Table 1 shows the tasks of Section 1.3 as a Gantt chart; a shaded cell means that the task was active in that week. The weeks are indicative and may be adjusted to the actual dates of the project.

Table 1: Project timeline (Gantt chart; columns are week numbers)

Task	1	2	3	4	5	6	7	8	9	10	11
Requirement analysis	#	#									
Design (architecture, data, UI)		#	#	#							
Data layer			#	#	#						
Quiz engine				#	#	#	#				
User interface					#	#	#	#	#		
Testing and fixing								#	#	#	
Documentation									#	#	#

1.5. Organization of the Report

The remainder of the report is organised as follows:

- **Chapter 1 (Introduction)** identifies the need, the problem and the tasks, and gives the timeline.
- **Chapter 2 (Design Flow / Process)** evaluates features, lists the design constraints, compares three alternative designs, justifies the selected design and presents the implementation plan with flowchart, architecture and algorithms.
- **Chapter 3 (Results Analysis and Validation)** describes the tools used, shows the implemented screens, and reports the testing, the defects found and the validation results.
- **Chapter 4 (Conclusion and Future Work)** summarises the outcome, discusses deviations from the expected results and suggests further improvements.
- The **References**, the **Appendices** (data formats, source excerpts and the complete test list), the **User Manual** and the **Achievements** follow the main chapters.

CHAPTER 2.

DESIGN FLOW / PROCESS

2.1. Evaluation and Selection of Specifications / Features

Before design began, similar tools were reviewed: console-based Java quiz programs that are commonly written as beginner exercises, general quiz platforms such as classroom-response and form-based quiz services, and mobile learning applications. From this review a list of candidate features was prepared and each feature was rated as Must, Should or Could according to how directly it solves the problem of Section 1.2 and how much effort it needs. Table 2 shows the result.

Table 2: Candidate features and their selection

ID	Feature	Priority	Decision
F1	Secure sign-in, registration and guest mode	Must	Selected
F2	Multiple topic categories (Java fundamentals, OOP, collections, data structures)	Must	Selected
F3	Multiple-choice questions with exactly four options and a stored correct answer	Must	Selected
F4	Instant right / wrong feedback with an explanation	Must	Selected
F5	Score, grade and breakdown (correct, wrong, skipped) at the end of a quiz	Must	Selected
F6	Per-question countdown timer	Should	Selected
F7	Lifelines: 50:50 and hint	Should	Selected
F8	Quiz history, statistics, day streak and recent-activity feed	Should	Selected
F9	Review of all answers after the quiz	Should	Selected
F10	Settings: display name, timer length, sound on/off, data reset	Could	Selected
F11	Keyboard shortcuts for answering	Could	Selected
F12	Online multiplayer / live leaderboard	Could	Rejected (needs a server)
F13	Database server for storage	Could	Rejected (setup cost)
F14	In-app question editor	Could	Deferred to future work

2.2. Design Constraints

The design was shaped by the constraints listed in Table 3. Constraints that are not relevant to a desktop software project are noted as such rather than ignored.

Table 3: Design constraints considered

Constraint	How it was considered in the design
Regulations / standards	Only the standard Java platform API is used, which keeps the project free of licensing issues. JSON files follow the ECMA-404 syntax [3]. Passwords are never stored in plain text, in line with the guidance on memorised secrets in NIST SP 800-63B [5].
Economic / cost	The whole stack (JDK, Swing, Git, an ordinary editor) is free. No server, database licence or hosting is needed, so the cost of building and running the application is zero.
Environmental	The application is a small local program with no network traffic and no server infrastructure, so its energy footprint is low. The dark interface also draws less power on OLED displays.
Health	A dark, low-glare colour scheme with high-contrast text reduces eye strain during long study sessions; text and background combinations were chosen with the contrast ideas of WCAG 2.1 in mind [10]. The timer length is configurable (15-60 s) so that time pressure can be reduced, and sound can be switched off.
Manufacturability / portability	For software this means that it must build and run easily on another machine. A single batch file builds the project into one executable JAR, and the application only needs a JDK 17 or newer.
Safety / security	Passwords are salted with 16 random bytes and hashed with SHA-256. User names are restricted to letters, digits, dot, dash and underscore because they become part of file names, which prevents invalid or malicious path characters. All data stays on the user's computer.
Professional	Code follows a layered (MVC-style) structure [6], uses meaningful names and Javadoc comments, and is versioned in Git [2]. The design takes account of known good practice for Java [1] and for user-interface usability [7].
Ethical	Scores are computed honestly and transparently; the application collects no personal data beyond a display name, shows no advertisements and does not track the user. Any AI assistance used in development is disclosed in Section 3.1.
Social	Keyboard shortcuts, readable font sizes, colour plus text/icon cues for right and wrong answers (not colour alone) and an offline mode make the tool usable by a wider group of students.
Political	No political constraint applies. The question content is purely technical and neutral.

2.3. Analysis and Feature Finalization Subject to Constraints

Applying the constraints of Section 2.2 to the features of Table 2 led to the following changes:

- **Removed:** online multiplayer and live leaderboard (F12) - they would need a server, an internet connection and the storage of other people's data, which conflicts with the cost, privacy and portability constraints. A database server (F13) was also removed for the same reason.

- **Modified:** persistence was changed from a database to human-readable JSON files read by a small custom parser. The 50:50 and hint lifelines were limited to **one use per quiz** so that they stay a strategic choice, and hints are disabled automatically on a question that has none.
- **Added:** a guest mode (no registration needed), a day-streak counter and recent-activity feed to encourage regular practice, a confirmation before leaving a running quiz, keyboard shortcuts and a sound on/off switch.
- **Deferred:** the in-app question editor (F14) is left for future work; questions are currently edited as JSON files.

2.4. Design Flow

Three alternative designs were prepared (Figure 2). All three give the student the same quiz functionality; they differ in the user-interface technology and in how data are stored.

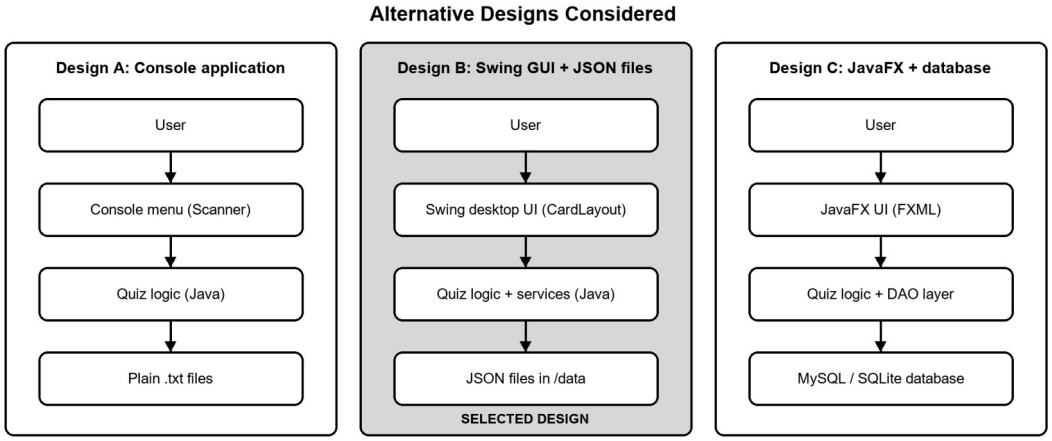


Figure 2: Block diagrams of the three alternative designs

2.4.1. Design A - Console application with text files

A command-line program that prints a question, reads the choice through the Scanner class and stores results in plain text files. It is the quickest design to write and has no graphical dependencies, but it offers no visual feedback, cannot show a countdown ring or score meter, has poor discoverability and is unattractive for repeated use.

2.4.2. Design B - Swing desktop GUI with JSON files

A single-window Swing application. Screens are Swing panels switched with a CardLayout; custom components are painted with Java 2D so that the application has a modern, consistent look. Data are stored as JSON files in a data folder. It needs only the JDK, runs offline and gives rich visual feedback.

2.4.3. Design C - JavaFX GUI with a database

A JavaFX interface described in FXML, with a data-access layer over MySQL or SQLite. This design is the most scalable and would allow queries over large amounts of data, but JavaFX is no longer bundled with the JDK, the database must be installed and configured, and the extra layers are too heavy for a one-semester project.

2.5. Design Selection

The designs were compared using five weighted criteria. Each criterion was given a score from 1 (poor) to 5 (excellent); the weighted score is the sum of weight x score divided by 100. Table 4 shows the comparison.

Table 4: Weighted comparison of the alternative designs

Criterion	Weight	Design A	Design B	Design C
User experience and visual feedback	30%	2	5	5
Development effort (fits the semester)	20%	5	4	2
Set-up and portability (zero install)	20%	5	5	2
Data handling and scalability	15%	1	3	5
Extensibility and maintainability	15%	2	4	4
Weighted score (out of 5)	**100%**	**3.05**	**4.35**	**3.65**

Design B scores highest and was selected. Its main advantages are the rich interface that makes the practice engaging, the fact that it needs nothing except the JDK, and an architecture that is simple enough to complete and test within the semester. Its main weakness - flat files do not scale to very large data sets - is acceptable because the data of a single student (a few thousand records at most) are small, and the storage code is isolated in one service class that can later be replaced by a database (Chapter 4).

2.6. Implementation Plan / Methodology

2.6.1. Architecture

The application uses a layered, MVC-style architecture (Figure 3). The **presentation layer** contains the window, the screens and reusable custom components; the **model layer** holds plain Java classes for questions, sessions, results and users; the **service layer** reads and writes files, parses JSON and plays sounds. Views never touch files directly; they ask a service. Table 5 lists the source packages.

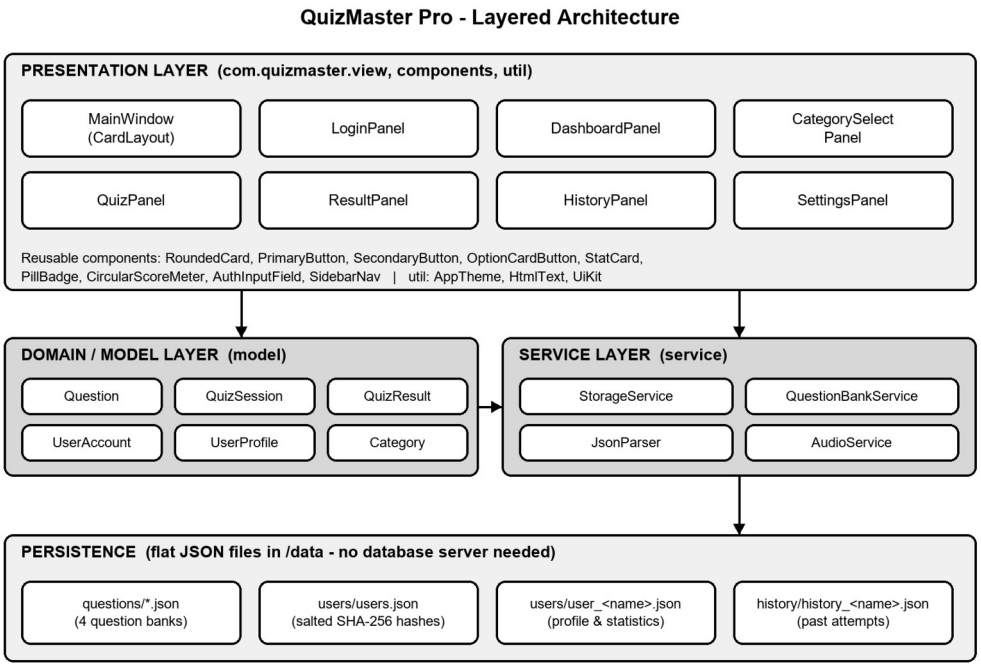


Figure 3: Layered architecture of QuizMaster Pro

Table 5: Source packages of the project

Package	Files	Lines	Responsibility
com.quizmaster.view	10	3,334	Main window, sidebar navigation and the seven screens / dialogs
com.quizmaster.components	8	1,174	Custom-painted reusable widgets (cards, buttons, badges, score ring, input field)
com.quizmaster.model	6	563	Question, QuizSession, QuizResult, UserAccount, UserProfile, Category
com.quizmaster.service	4	938	StorageService, QuestionBankService, JsonParser, AudioService
com.quizmaster.util	4	283	AppTheme (design tokens), HtmlText, DarkScrollBarUI, UiKit
com.quizmaster	1	54	Main.java - entry point: look-and-feel, data folder lookup, window start-up
Total	**33**	**6,346**	

2.6.2. Flowchart of a quiz session

Figure 4 shows the flow from start-up to the saved result. After sign-in the user chooses a category and the number of questions; the questions are shuffled and shown one at a time with a countdown. A question ends when the answer is confirmed or the question is skipped (or the timer runs out); the loop continues until no question is left, after which the result is saved and the score screen is shown.

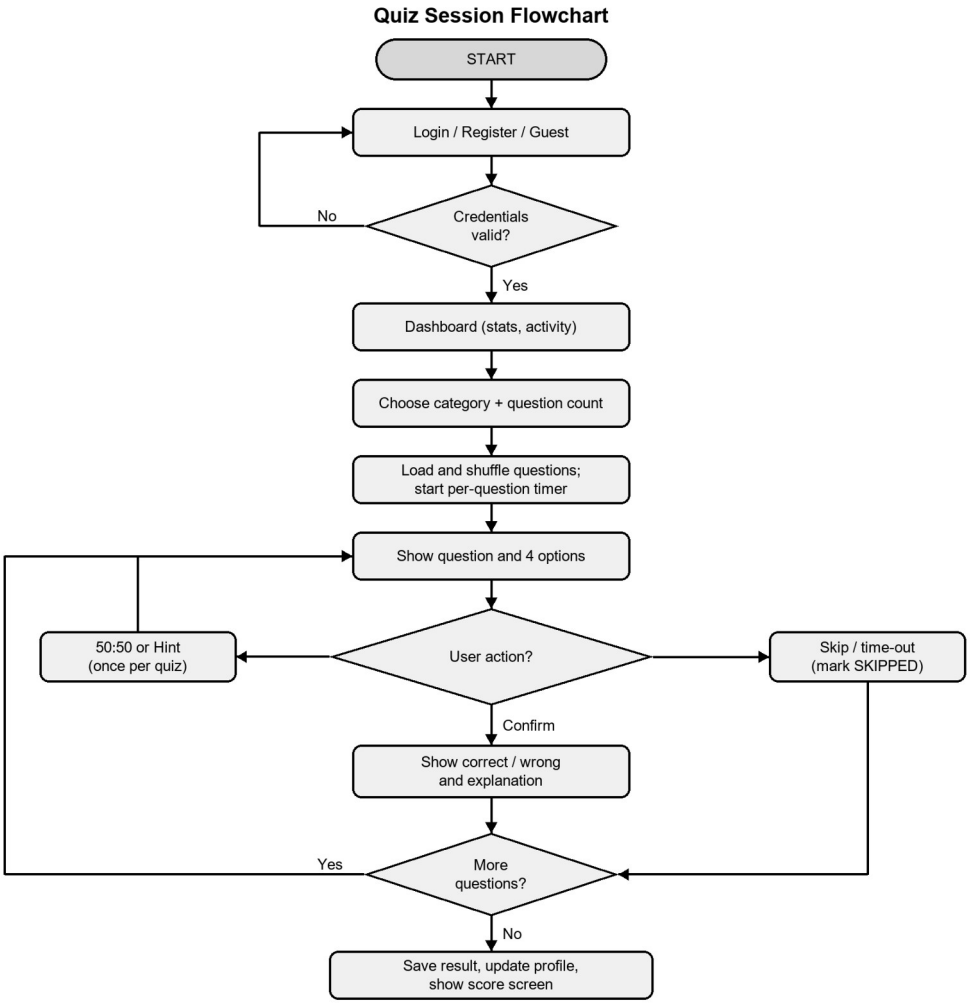


Figure 4: Flowchart of a quiz session

2.6.3. Class design

Figure 5 shows the most important classes and their relationships. MainWindow owns the screens and creates a QuizSession when a quiz starts. QuizSession owns the list of Question objects and the answer state and can build a QuizResult. StorageService manages accounts, profiles and history, while QuestionBankService loads the question files.

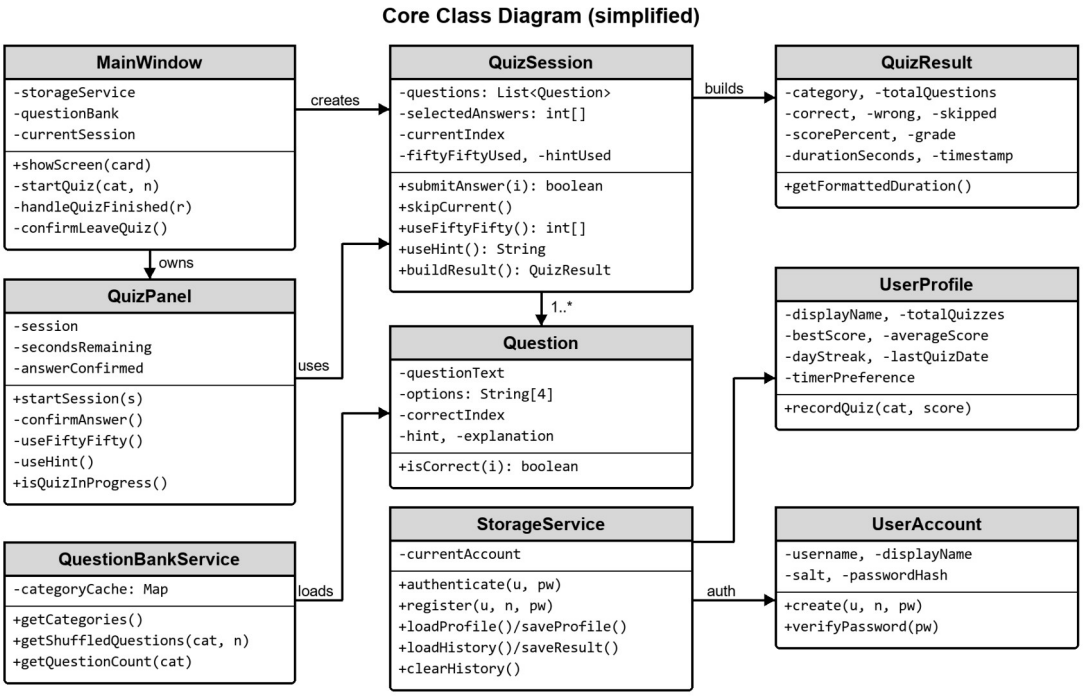


Figure 5: Simplified class diagram

2.6.4. Key algorithms

Scoring and grading. With C correct answers out of N questions, the score is S = floor(C x 100 / N) per cent. The grade is A+ for S >= 95, A for S >= 80, B for S >= 65, C for S >= 50 and F below 50. Skipped and unanswered questions earn no marks and are counted separately.

Running average. After the n-th completed quiz with score s, the stored average becomes avg = floor((avg x (n - 1) + s) / n). Best score is the maximum of all scores.

Day streak. The date of the last completed quiz is stored. If today equals that date the streak is unchanged; if it equals yesterday the streak is increased by one; otherwise it restarts at one.

50:50 lifeline. The three wrong option indices are collected and shuffled, and the first two are removed. The correct option is therefore never removed, and the lifeline can be used once per quiz.

Password storage. On registration a random 16-byte salt is generated; the stored hash is SHA-256(salt + password). On sign-in the same computation is repeated and compared with the stored hash.

The pseudo-code of the main quiz loop is given below.

```
load questions(category) -> shuffle -> take first n
session = new QuizSession(questions, timerSeconds)
for each question q in session:
    show q; secondsRemaining = timerSeconds; start timer
    wait until one of:
        user selects option and confirms  -> session.submitAnswer(option)
        user presses Skip                 -> session.skipCurrent()
        timer reaches 0                   -> submit selection, else skip
    show correct/wrong colours and explanation
result = session.buildResult()      // counts, score, grade, duration
storage.saveResult(result)          // newest first, max 50 kept
profile.recordQuiz(category, score) // totals, best, average, streak
show result screen
```

2.6.5. Data design

All data are JSON files below the data folder: **questions/** (one file per category), **users/users.json** (accounts), **users/user_<name>.json** (profile and statistics) and **history/history_<name>.json** (past attempts, newest first, at most 50). A small custom JSON parser (about 300 lines) reads and writes these files, so the application has no external library. Sample file contents are given in Appendix-1.

CHAPTER 3.

RESULTS ANALYSIS AND VALIDATION

3.1. Implementation of Solution

The solution was implemented with the tools in Table 6. Every tool is free; the application itself uses only the standard Java platform and no third-party library.

Table 6: Tools and technologies used

Purpose	Tool / technology
Programming language and runtime	Java SE (JDK 17 or newer; developed and tested on JDK 26)
User interface	Java Swing and AWT; custom painting with java.awt.Graphics2D (anti-aliased shapes, arcs, gradients)
Audio feedback	javax.sound.sampled - tones are generated in code (sine and triangle waves), so no audio files are needed
Data storage	JSON files read and written by a custom parser (JsonParser); java.nio.file for file access
Security	java.security.MessageDigest (SHA-256) and SecureRandom for salts
Editor and build	Visual Studio Code, javac / jar command-line tools and Windows batch scripts (build.bat, run.bat)
Version control	Git and GitHub (public repository: github.com/SHZunknown11/quizmaster-pro)
Testing	Plain Java test programs (logic tests, end-to-end navigation test driving the real window, off-screen rendering of every screen for visual inspection)
Diagrams and report	Python (Pillow) for diagrams, LibreOffice / Word for the report
AI assistance	Claude Code (an AI coding assistant) was used during development for UI/UX bug review and fixing, test creation and drafting of this report. All results were reviewed by the authors. Edit or remove this row to match your university's policy on AI assistance.

3.2. Screens of the Implemented Application

This section shows each screen exactly as rendered by the application (screenshots at the default window size). The design uses a deep slate background with a warm amber accent, a 72-pixel navigation rail on the left and card-based content.

3.2.1. Login and registration

The login screen offers three ways in: sign in with an account, create an account, or continue as a guest. Fields show a focus glow, the password has a show/hide toggle and errors appear

in a dismissible banner. A quick-fill link and hint line make the demonstration account easy to use.

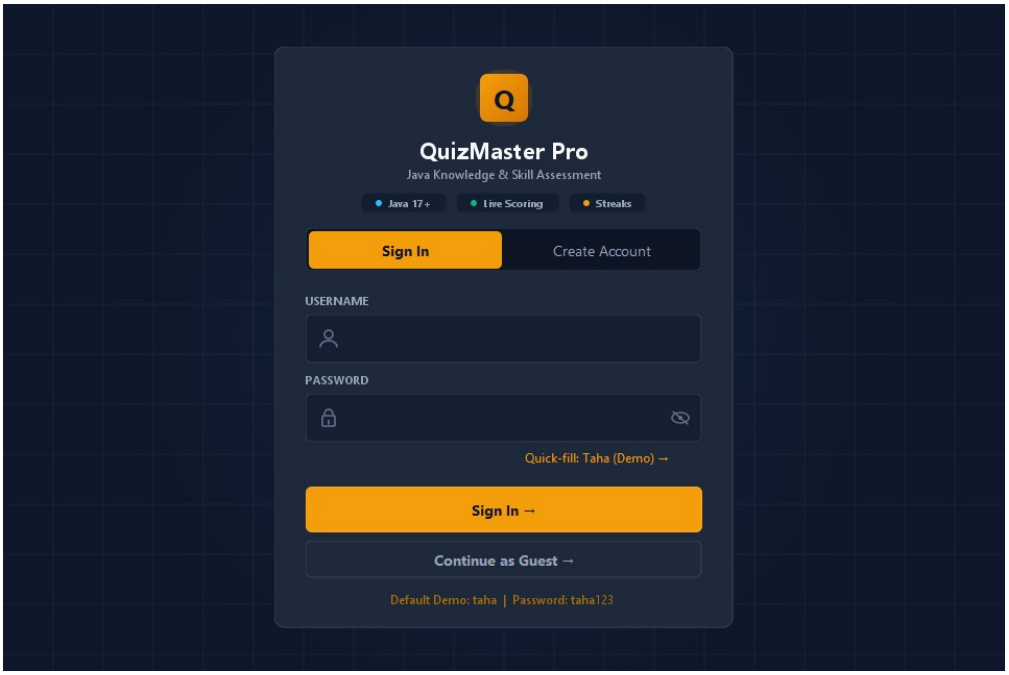


Figure 6: Login screen

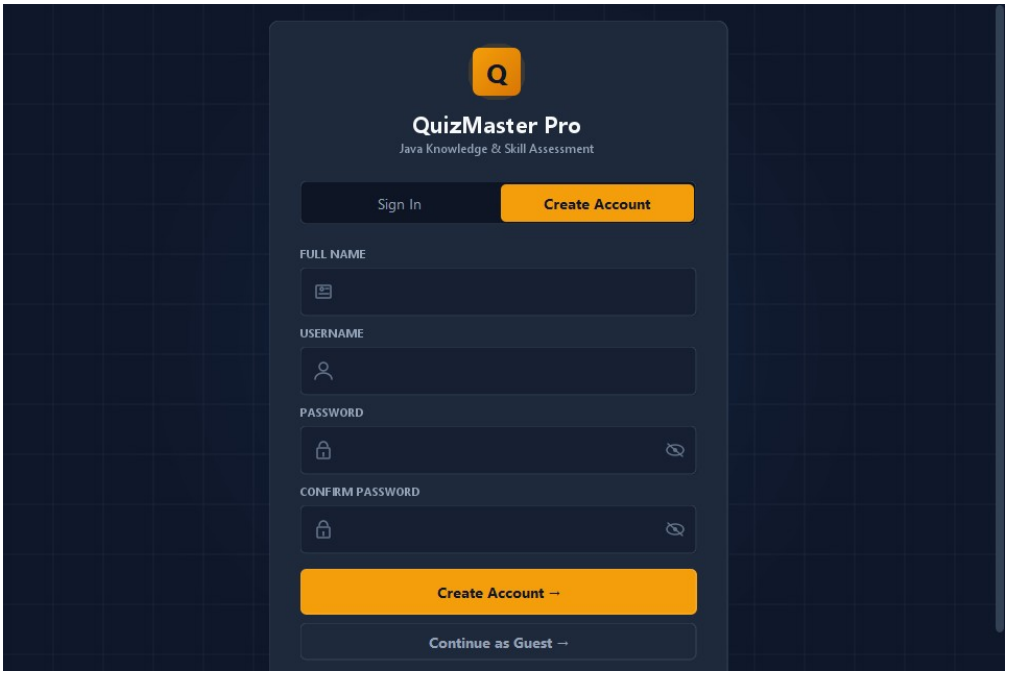


Figure 7: Create-account screen with validation

3.2.2. Dashboard

The dashboard greets the user according to the time of day and shows four statistics (total quizzes, best score, day streak and average score), the recent-activity feed and a prominent button to start a new quiz.

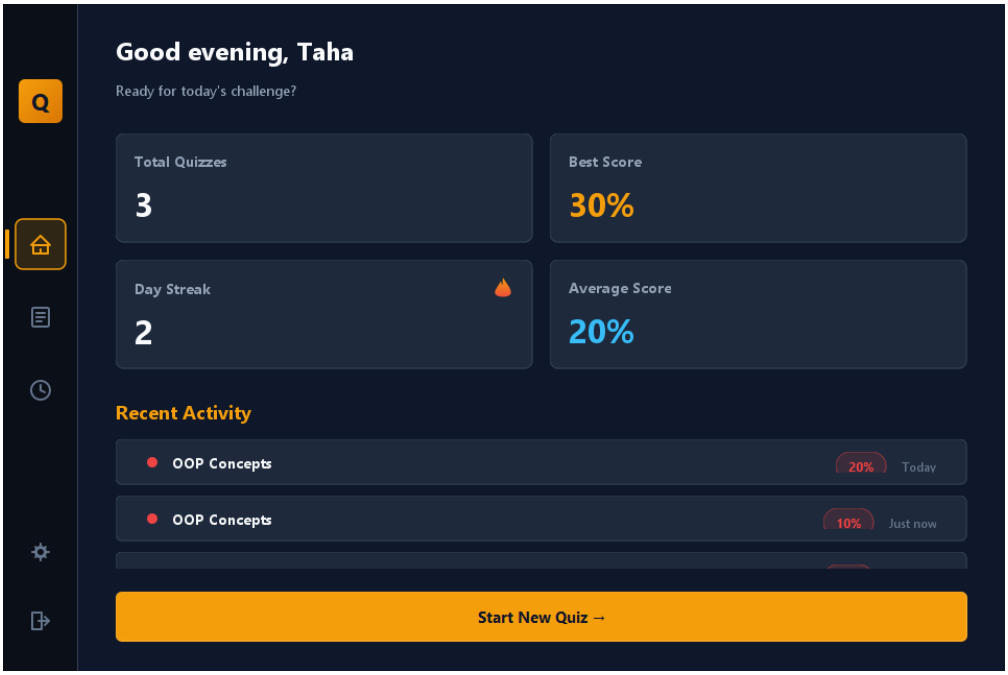


Figure 8: Dashboard with statistics and recent activity

3.2.3. Category selection

Four topic cards show the number of available questions and a short description. The user also chooses 5, 10 or 15 questions before starting.

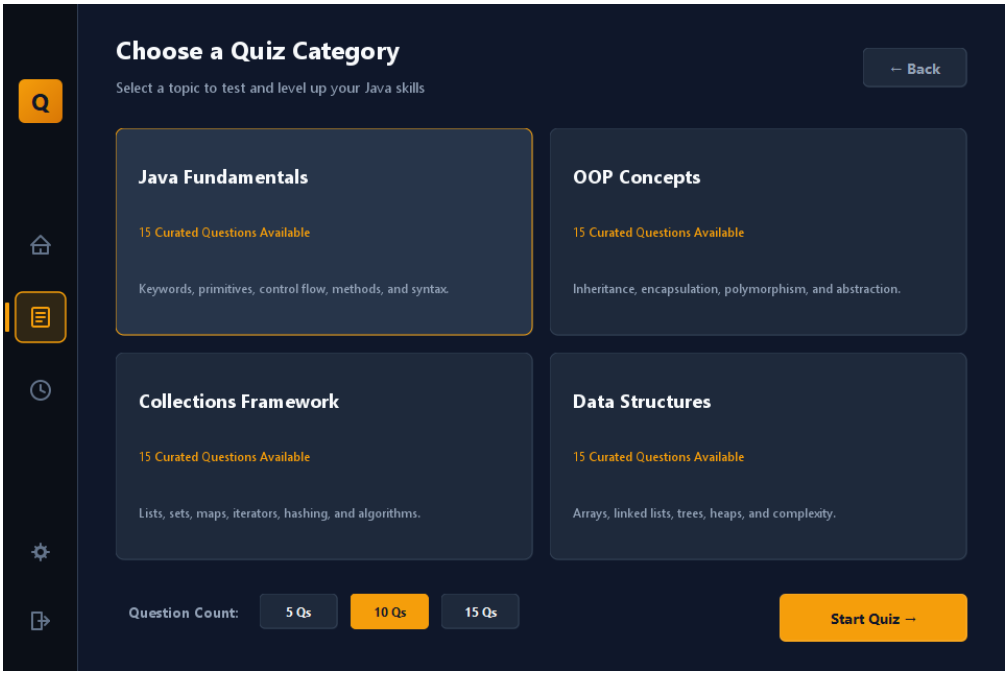


Figure 9: Category and question-count selection

3.2.4. Quiz screen

The quiz screen shows the category and question number, a progress bar, a circular countdown timer, the question, four option cards and the lifelines. Options can be chosen with the mouse or with the keys 1-4 / A-D, and Enter confirms the answer.

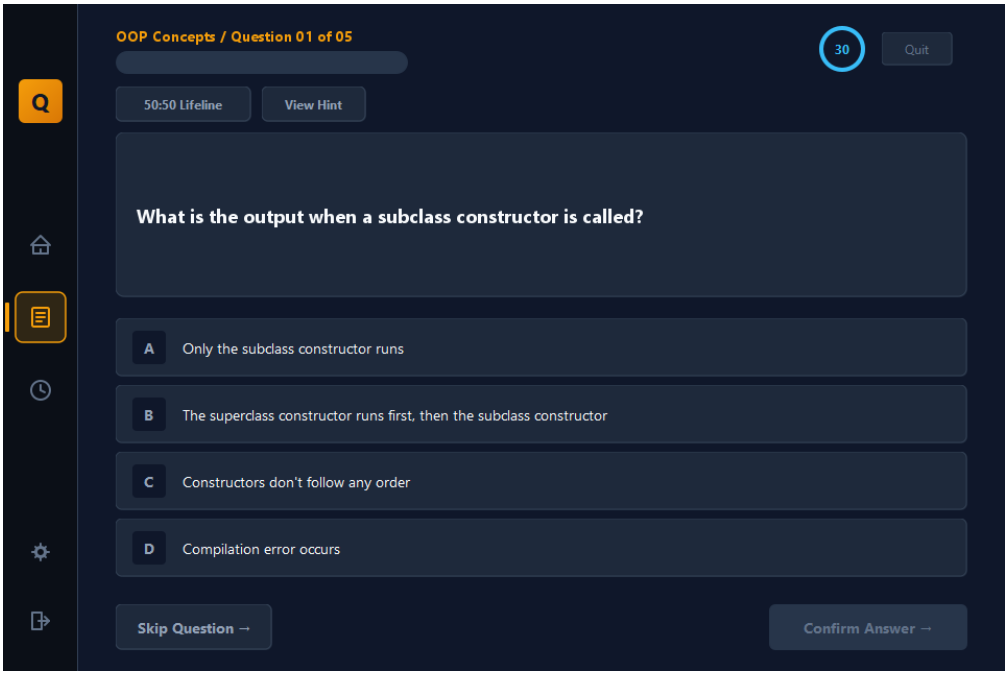


Figure 10: Quiz screen with timer and four options

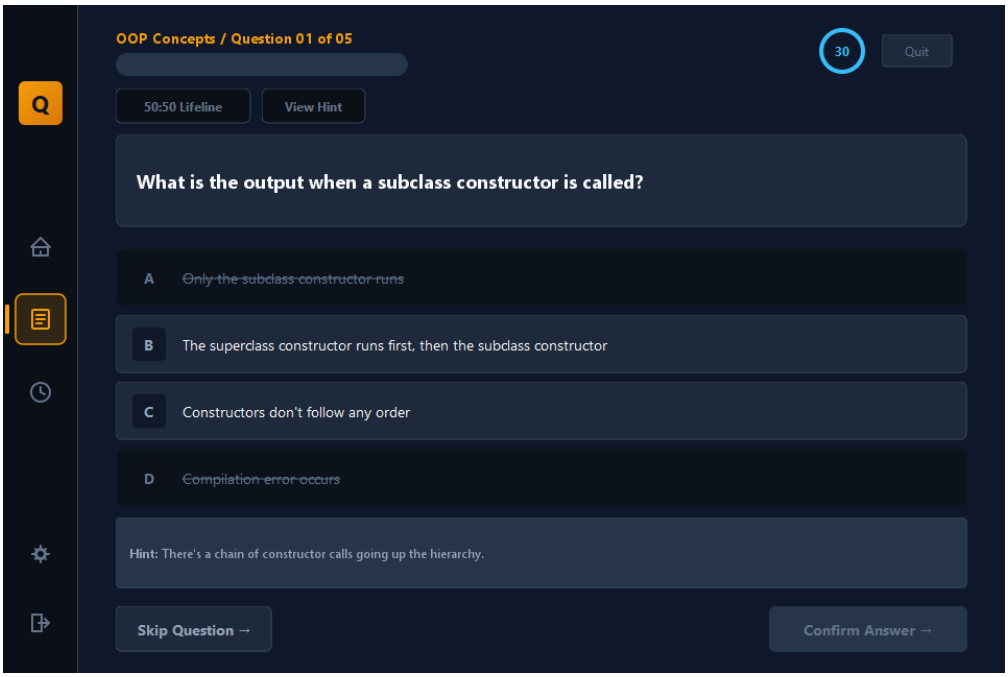


Figure 11: After using the 50:50 lifeline and the hint

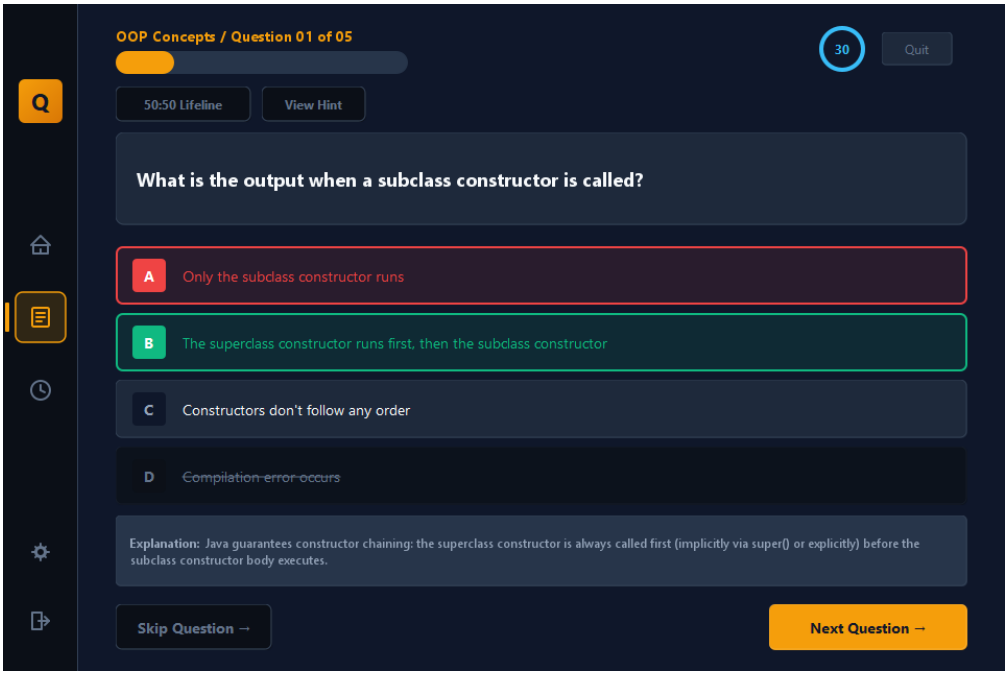


Figure 12: Answer feedback: correct option in green, wrong choice in red, with explanation

3.2.5. Result and review

At the end the score is shown on an animated ring together with the numbers of correct, wrong and skipped questions and the grade. The user can try the same quiz again, start a new one, or open the question-by-question review.

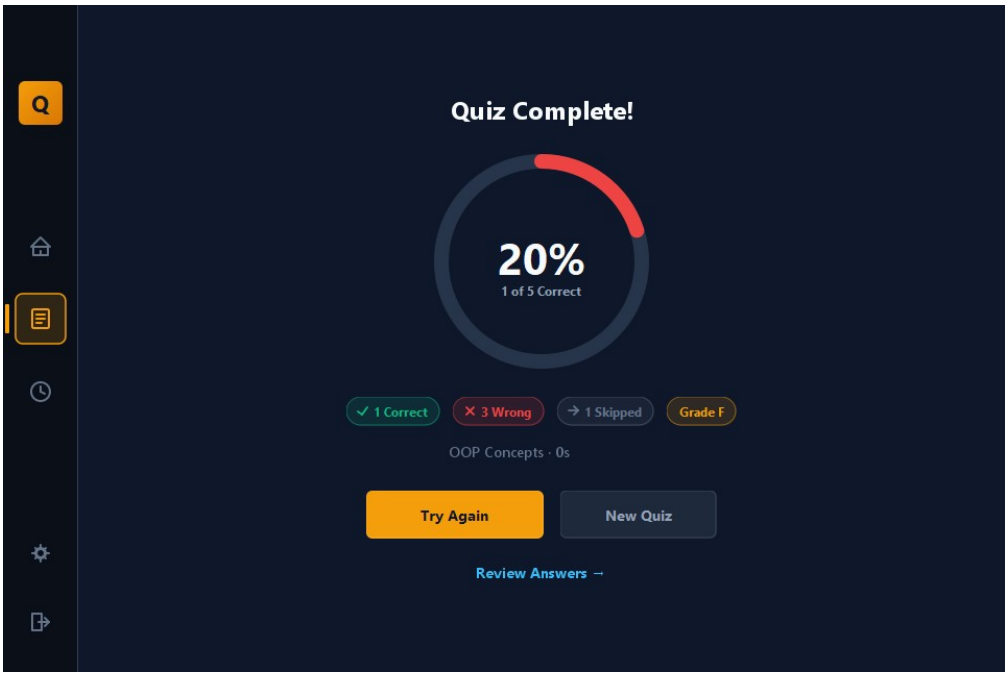


Figure 13: Result screen with score ring, breakdown and grade

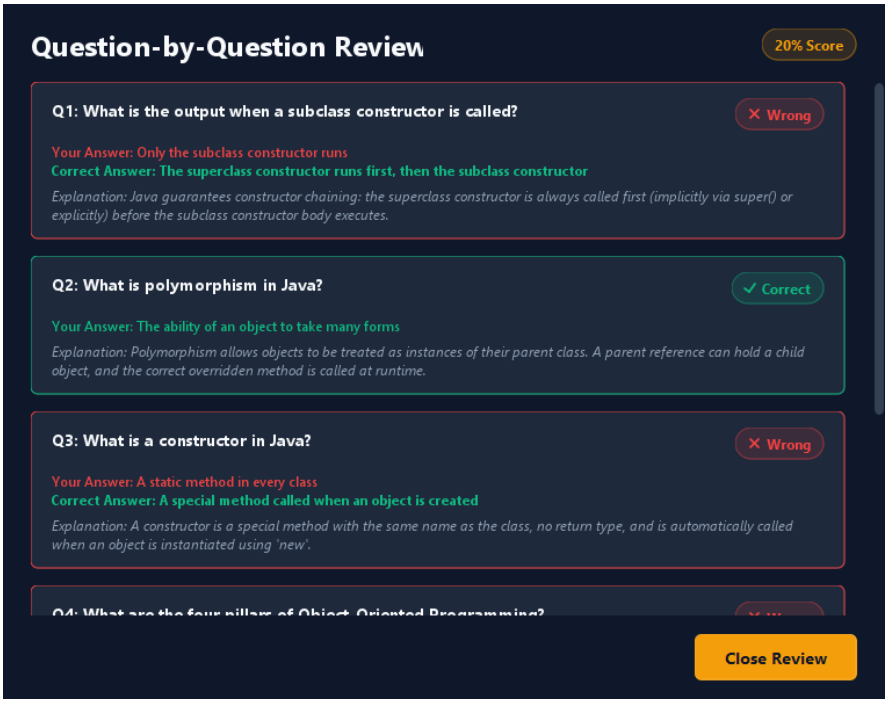


Figure 14: Question-by-question review dialog

3.2.6. History and settings

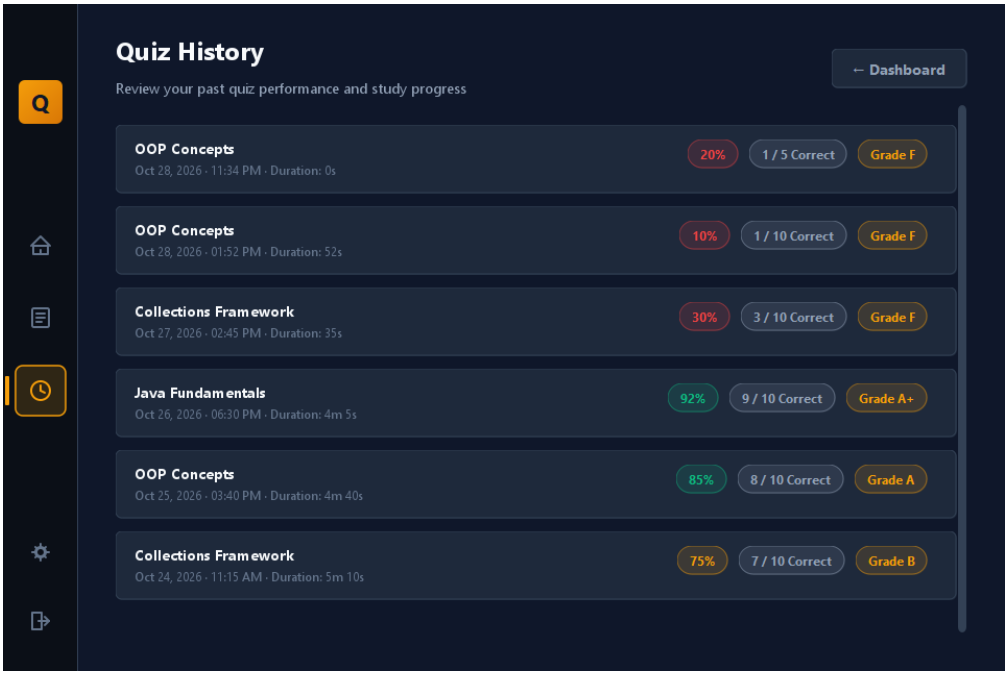


Figure 15: Quiz history with score, grade and duration

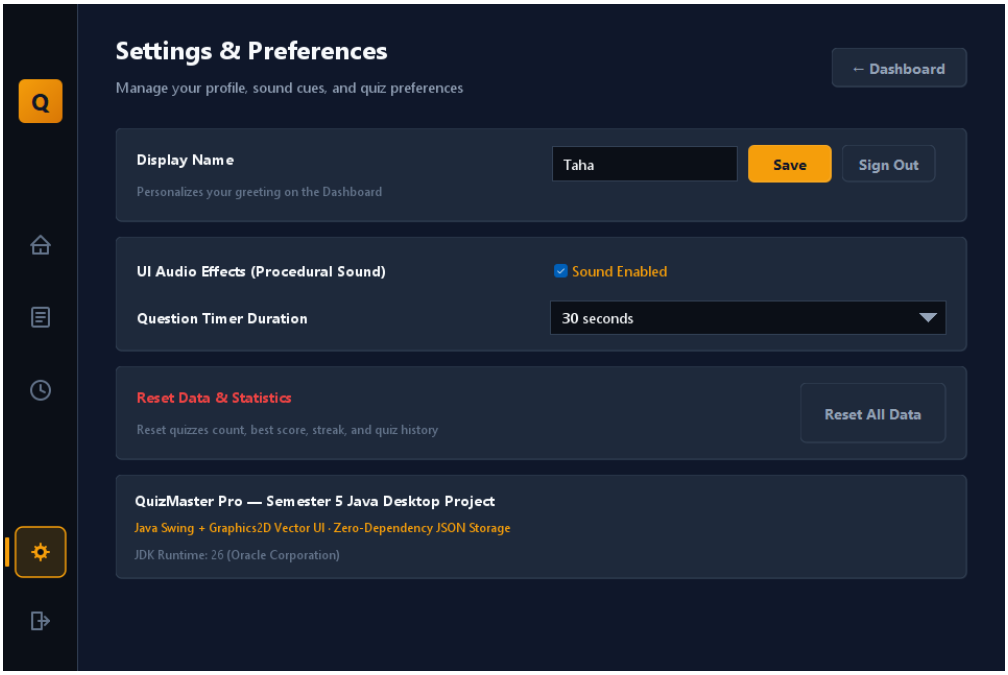


Figure 16: Settings: display name, sound, timer length, data reset

3.3. Testing, Characterization and Data Validation

Testing combined four complementary methods. First, **automated logic tests** (37 checks) exercise the model and service classes without a window. Second, an **end-to-end test** drives the real main window and the confirmation dialogs. Third, every screen was **rendered off-screen** at the minimum window size and inspected. Fourth, a **clean-clone test** checked that a fresh copy of the repository builds and starts on a machine that has only the JDK.

3.3.1. Automated logic tests

Table 7 summarises the 37 logic checks; the complete list with descriptions is in Appendix-2. All of them passed. The tests use a temporary data folder, so they never touch real user data.

Table 7: Summary of automated logic tests

Group	Test IDs	Tests	Passed	Failed
Grading and scoring	T01-T08	8	8	0
Quiz session and lifelines	T09-T13	5	5	0
Profile statistics and streak	T14-T18	5	5	0
Accounts and security	T19-T26	8	8	0
Persistence (history, JSON)	T27-T31	5	5	0
Question bank	T32-T36	5	5	0
Text safety	T37	1	1	0
Total		**37**	**37**	**0**

Several results are worth highlighting: the 50:50 lifeline was run 2,000 times with random correct positions and never removed the correct option (T09); grade boundaries behave exactly at 95, 80, 65, 50 and 49 per cent (T01-T05); a quiz with no questions yields 0 per cent without a division error (T07); the stored hash is a 64-character SHA-256 value that does not contain the password, and the same password gets a different salt and hash in each account (T20, T21); and a JSON round trip preserves quotes, new lines, back-slashes and Unicode (T31).

3.3.2. End-to-end navigation test

The real MainWindow was started, a quiz was begun and the sidebar was used to leave it. A helper thread answered the confirmation dialogs automatically. Table 8 shows the checks; all passed.

Table 8: End-to-end checks on the real application window

ID	Check	Result
E1	Start a quiz, then check that it is reported as in progress	Pass
E2	Click another sidebar item during a quiz: a confirmation dialog appears	Pass
E3	Answer "No": the quiz stays in progress	Pass
E4	Answer "No": the sidebar highlight returns to "Quizzes"	Pass
E5	Click another item again: the confirmation appears again	Pass
E6	Answer "Yes": the quiz ends and the target screen opens	Pass
E7	Navigate with no quiz running: no dialog is shown	Pass
E8	Save a result, then check it is in the history	Pass
E9	Clear the history: the history becomes empty	Pass
E10	Register a new account	Pass
E11	New account starts with a streak of 0	Pass

3.3.3. Visual and clean-clone validation

All eight screens and the review dialog were rendered at a content size of 848 x 600 pixels, which is smaller than the default window, and checked for clipped text, overlapping elements, unreadable colours and missing glyphs. The defects found this way are listed in Section 3.4. A fresh copy of the GitHub repository was cloned into an empty folder, compiled with javac, packaged with jar and started; the application launched and created its demonstration account automatically.

3.4. Defects Found and Corrected

The review and testing described above uncovered 23 defects, which were corrected. Table 9 lists them by type.

Table 9: Defects found during testing and their corrections

ID	Type	Defect	Correction
B1	Rendering	Check, cross, refresh and emoji characters (for example in the result pills, the Quit button and Try Again) showed as empty boxes because the UI font lacks them.	Replaced by vector-drawn icons in PillBadge and plain text on buttons.
B2	Rendering	Question text such as List<String> lost the part inside the angle brackets because it was placed unescaped into HTML labels.	Added HtmlText.escape(); all question, option, hint and explanation text is escaped.
B3	Rendering	Fixed-width HTML text wrapped wider than its card on scaled (125 %) displays because Swing scales CSS pixels by screen DPI.	Wrap widths are converted for the screen DPI and re-computed when the window is resized.
B4	Rendering	Long answer options were drawn past the edge of the option card.	Options now wrap to two lines and are ellipsised with the full text in a tooltip.
B5	Rendering	Review dialog overflowed horizontally and showed raw code fences (```).	Fixed widths, code shown in a monospace block, horizontal scroll removed, Esc closes the dialog.
B6	Rendering	Settings timer drop-down showed white text on a white background.	Custom dark combo-box UI and renderer; entries read "30 seconds".
B7	Rendering	The About card text in Settings was clipped.	Card height and row spacing corrected.
B8	Rendering	Bright white system scroll bars clashed with the dark theme.	Added DarkScrollBarUI and applied it to every scroll pane.

ID	Type	Defect	Correction
B9	Layout	Back buttons stretched to the height of the two-line page title.	Wrapped in a vertically centred panel (UiKit).
B10	Layout	Large uneven gaps on the result screen.	Fixed maximum heights and centred the content vertically.
B11	Layout	Quiz content was clipped on short windows; the explanation could be hidden below the fold.	Quiz centre area is scrollable and the explanation scrolls into view automatically.
B12	Layout	Lifeline buttons were indented 10 px relative to the question card.	Removed the FlowLayout gap and used an explicit spacer.
B13	Data display	Older history entries showed a raw "2026-10-26 18:30" date; the dashboard showed raw ISO dates.	Parser accepts several formats; dashboard uses relative dates ("Today", "Yesterday").
B14	Data display	Category cards appeared in arbitrary directory order.	Categories are sorted into a fixed, sensible order.
B15	Behaviour	Leaving a running quiz via the sidebar or sign-out silently discarded progress.	A confirmation dialog is shown; the timer is paused while it is open and the sidebar highlight is restored on cancel.
B16	Behaviour	Using 50:50 could leave an already-selected, now eliminated option selected, so it could still be submitted.	Selection is cleared when the selected option is eliminated.
B17	Behaviour	A question without a hint consumed the one-per-quiz hint lifeline and showed an empty hint.	Hint button is disabled for such questions and the lifeline is not consumed.
B18	Behaviour	Quiz key bindings could fire while another screen was visible, and a double Enter could save a result twice.	Bindings check that the quiz is showing; completion is guarded by a flag.
B19	Behaviour	The progress bar never reached 100 % and showed a dot at 0 %.	Progress counts the answered question; nothing is drawn at 0 %.
B20	Data	"Reset All Data" did not clear the history it promised to clear.	Added StorageService.clearHistory().
B21	Data	A new account showed a 1-day streak with zero quizzes.	New accounts start at 0; the first quiz sets the streak to 1.
B22	Input	Passwords were silently trimmed; user names were not validated although they become file names.	Passwords are kept as typed; user names must be 3-20 characters of letters, digits, dot, dash or underscore.
B23	Usability	Settings silently ignored an empty display name; keyboard focus was invisible; no cursor in the user-name box at start-up.	Clear warning messages, visible focus ring on Tab navigation, automatic initial focus.

3.5. Interpretation of Results

All 37 logic checks and all 11 end-to-end checks passed after the corrections, and every screen renders without clipping at the minimum window size. The most valuable finding of the validation was that several defects were **invisible in the source code but obvious on screen** - for example the missing glyphs, the white drop-down and the DPI-dependent text wrapping - which shows the importance of rendering and inspecting the interface and not only testing the logic. Equally, defects such as the quiz being abandoned without warning or the reset button not clearing history were logic or behaviour problems that only appeared when the application was used the way a student would use it.

The size of the delivered software is 33 classes and 6,346 lines of Java, together with a question bank of 60 questions (19 easy, 26 medium and 15 hard) spread over four categories. A student needs only a JDK to build and run it.

CHAPTER 4.

CONCLUSION AND FUTURE WORK

4.1. Conclusion

The project set out to give Java students an offline, distraction-free quiz tool with instant feedback and visible progress. The finished application meets this objective: it provides secure sign-in, four topic categories with 60 questions, a per-question timer, 50:50 and hint lifelines, explanations, a graded result with review, a persistent history with statistics, and configurable settings, all in a consistent, modern interface that runs from a single JAR file. The layered architecture separates the screens, the model and the storage, which made it possible to test the logic without a window and to fix interface defects without touching the data code.

Expected results. The expected outcome was a working quiz application that a student can build and use without installing anything but the JDK. This was achieved and verified by the clean-clone test.

Deviations from the expected results and their reasons.

- The application stores data in JSON files instead of a database. This was a deliberate decision (Section 2.5) to remove set-up effort; the consequence is that complex queries and very large data sets are not supported.
- Online multiplayer and leaderboards were not implemented because they would require a server and the storage of other users' data (Section 2.3).
- The look of the interface depends on the Segoe UI font that ships with Windows; on other operating systems a substitute font is used and spacing can differ slightly.
- The question bank is limited to 60 questions; a larger bank would give more variety between attempts.
- Passwords are hashed with salted SHA-256, which is adequate for a local study tool but is not a deliberately slow password-hashing function such as bcrypt or Argon2 as would be needed for an internet-facing service.

4.2. Future Work

The following modifications and extensions are suggested as the way ahead:

- **Question editor and import/export** - add or edit questions inside the application and import question sets from CSV or JSON files.
- **Larger and adaptive question bank** - more topics (exception handling, multithreading, streams, JDBC) and difficulty that adapts to the user's performance.
- **Progress analytics** - charts of score over time and per-topic strengths and weaknesses.
- **Database back end** - replace the file storage by SQLite behind the existing StorageService interface to support larger data and multiple devices.
- **Automated unit-test suite** - convert the current test programs into a JUnit suite run by a build tool such as Maven or Gradle, and add continuous integration on GitHub.
- **Accessibility and portability** - a high-contrast and light theme, scalable fonts, full keyboard navigation and a native installer created with jpackage.
- **Stronger password hashing** - use PBKDF2 or Argon2 for stored passwords.

REFERENCES

1. Bloch, J. (2018) *Effective Java*. 3rd edn. Boston: Addison-Wesley.
2. Chacon, S. and Straub, B. (2014) *Pro Git*. 2nd edn. New York: Apress.
3. ECMA International (2017) *ECMA-404: The JSON Data Interchange Syntax*. 2nd edn. Geneva: Ecma International.
4. Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
5. Grassi, P. A., Garcia, M. E. and Fenton, J. L. (2017) *Digital Identity Guidelines: Authentication and Lifecycle Management*. NIST Special Publication 800-63B. Gaithersburg, MD: National Institute of Standards and Technology.
6. Krasner, G. E. and Pope, S. T. (1988) 'A cookbook for using the model-view-controller user interface paradigm in Smalltalk-80', *Journal of Object-Oriented Programming*, 1(3), pp. 26-49.
7. Nielsen, J. (1994) *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group.
8. Oracle (n.d.) *The Java Tutorials: Creating a GUI With Swing*. Oracle Corporation. Available at: https://docs.oracle.com/javase/tutorial/uiswing/
9. Oracle (n.d.) *Java Platform, Standard Edition API Specification* (java.awt.Graphics2D, javax.swing, javax.sound.sampled, java.security.MessageDigest). Oracle Corporation. Available at: https://docs.oracle.com/en/java/javase/
10. W3C (2018) *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium.

APPENDIX

Appendices of the project report "QuizMaster Pro: A Java Swing Desktop Quiz Application"

A1. Appendix-1: Project Structure and Data Formats

The repository layout is shown below.

```
quizmaster-pro/
  src/com/quizmaster/
    Main.java             entry point
    view/         (10)    MainWindow, SidebarNav, LoginPanel,
                          DashboardPanel, CategorySelectPanel,
                          QuizPanel, ResultPanel, HistoryPanel,
                          SettingsPanel, ReviewAnswersDialog
    components/   (8)     RoundedCard, PrimaryButton, SecondaryButton,
                          OptionCardButton, StatCard, PillBadge,
                          CircularScoreMeter, AuthInputField
    model/        (6)     Question, QuizSession, QuizResult,
                          UserAccount, UserProfile, Category
    service/      (4)     StorageService, QuestionBankService,
                          JsonParser, AudioService
    util/         (4)     AppTheme, HtmlText, DarkScrollBarUI, UiKit
  data/
    questions/    java_fundamentals.json, oop_concepts.json,
                  collections_framework.json, data_structures.json
    users/        users.json, user_<name>.json    (created at run time)
    history/      history_<name>.json             (created at run time)
  build.bat   run.bat   README.md
```

A question in a question-bank file:

```
{
  "id": "jf01",
  "difficulty": "EASY",
  "questionText": "Which keyword is used to define a class in Java?",
  "options": ["struct", "class", "define", "object"],
  "correctOptionIndex": 1,
  "hint": "It's the same keyword used in C++ and C#.",
  "explanation": "The 'class' keyword is used to declare a class in Java."
}
```

An account record in users.json (the password itself is never stored):

```
{
  "username": "taha",
  "displayName": "Taha",
  "salt": "0404db00d03ccd18c2545ff6e42a8fc9",
  "passwordHash": "c03066ed3e540ec24ca2570a979bc107...cc128fb33",
  "createdAt": "2026-10-27T14:59:57.981772900"
}
```

A history record in history_<name>.json:

```
{
  "id": "23393446-eb29-4856-8a02-01792081867a",
  "category": "OOP Concepts",
  "totalQuestions": 10, "correct": 1, "wrong": 1, "skipped": 8,
  "scorePercent": 10, "durationSeconds": 52, "grade": "F",
  "timestamp": "2026-10-28T13:52:27.772375400"
}
```

A2. Appendix-2: Selected Source Code

The 50:50 lifeline from QuizSession.java. The correct option is never added to the list of candidates for removal:

```
public int[] useFiftyFifty() {
    if (fiftyFiftyUsed) return null;
    fiftyFiftyUsed = true;

    Question q = getCurrentQuestion();
    List<Integer> wrongIndices = new ArrayList<>();
    for (int i = 0; i < 4; i++) {
        if (i != q.getCorrectIndex()) wrongIndices.add(i);
    }
    Collections.shuffle(wrongIndices);   // pick 2 random wrong ones
    return new int[]{ wrongIndices.get(0), wrongIndices.get(1) };
}
```

Escaping text before it is placed in a Swing HTML label (HtmlText.java), which fixes the loss of generic types such as List<String>:

```
public static String escape(String s) {
    if (s == null) return "";
    return s.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
            .replace("\r\n", "\n")
            .replace("\n", "<br>");
}
```

Grade calculation from the percentage score (QuizResult.java):

```
if (scorePercent >= 95) grade = "A+";
else if (scorePercent >= 80) grade = "A";
else if (scorePercent >= 65) grade = "B";
else if (scorePercent >= 50) grade = "C";
else grade = "F";
```

A3. Appendix-3: Complete List of Automated Logic Tests

The tests are run by a plain Java program against the compiled classes. Output of the final run: 37 tests, 37 passed, 0 failed.

Table 10: Automated logic tests and results

ID	Test description	Result
T01	grade A+ at 95%	Pass
T02	grade A at 80% and 94%	Pass
T03	grade B at 65%	Pass
T04	grade C at 50%	Pass
T05	grade F at 49%	Pass
T06	score is floor(2/3*100)=66	Pass
T07	empty quiz gives 0%, no divide-by-zero	Pass
T08	duration formats as '4m 5s' and '52s'	Pass

ID	Test description	Result
T09	50:50 removes exactly two distinct WRONG options (2000 trials)	Pass
T10	50:50 can only be used once per quiz	Pass
T11	hint returns text once, then null	Pass
T12	correct answer scores CORRECT, wrong scores WRONG	Pass
T13	wrong=1 skipped=1 unanswered=1 correct=1 counted separately	Pass
T14	recordQuiz updates total, best and average (80,60 -> 2, 80, 70)	Pass
T15	recent activity capped at 5, newest first	Pass
T16	quiz on consecutive day extends streak (3 -> 4)	Pass
T17	gap of several days resets streak to 1	Pass
T18	second quiz on the same day keeps streak unchanged	Pass
T19	correct password verifies, wrong password is rejected	Pass
T20	same password gets different salt and hash per account	Pass
T21	stored hash is 64 hex chars (SHA-256) and not the plain password	Pass
T22	demo account 'taha' is seeded on first launch	Pass
T23	register creates account and signs in	Pass
T24	duplicate username is rejected (case-insensitive)	Pass
T25	login is case-insensitive on username, case-sensitive on password	Pass
T26	wrong password does not sign in	Pass
T27	saved result is read back identically (category, score, grade)	Pass
T28	history is capped at the 50 most recent attempts	Pass
T29	clearHistory empties the history	Pass
T30	users have separate histories	Pass
T31	JSON round-trip preserves quotes, newline, backslash and unicode	Pass
T32	4 categories load in the fixed display order	Pass
T33	every category contains 15 questions (60 in total)	Pass
T34	requesting 5 / 10 questions returns exactly that many, 99 returns all 15	Pass
T35	all 60 questions have 4 options and a valid correct index	Pass
T36	question ids are unique across the bank	Pass
T37	HtmlText escapes generics and ampersands	Pass

A4. Appendix-4: Source Code Repository

The complete source code, question banks, build scripts and README are available in the public repository: **https://github.com/SHZunknown11/quizmaster-pro**.

USER MANUAL

(Complete step-by-step instructions to run and use the project)

M1. Requirements

- A computer with **Windows** (recommended; Linux and macOS also work, see M3).
- A **JDK 17 or newer** (a JRE alone is not enough because the project is compiled on your machine). Check with the command `javac -version`.
- **Git** to download the project (or download the ZIP file from GitHub).

M2. Download, Build and Run (Windows)

1. Open a terminal (Command Prompt or PowerShell).
2. Download the project: `git clone https://github.com/SHZunknown11/quizmaster-pro.git`
3. Go into the folder: `cd quizmaster-pro`
4. Build it by double-clicking **build.bat** (or run it in the terminal). It compiles the code into the bin folder and creates **QuizMasterPro.jar**. Press a key when it says "Build complete".
5. Start the program by double-clicking **run.bat**, or run `java -jar QuizMasterPro.jar`.
6. Always start the program from the project folder, because it looks for the data folder next to it.

M3. Run on Linux or macOS

```
git clone https://github.com/SHZunknown11/quizmaster-pro.git
cd quizmaster-pro
mkdir -p bin
javac -encoding UTF-8 -d bin -sourcepath src src/com/quizmaster/Main.java
java -cp bin com.quizmaster.Main
```

M4. Signing In

1. On the login screen enter a user name and password and press **Sign In** (or Enter). For a quick demonstration use the account **taha / taha123**; the "Quick-fill" link fills it in for you.
2. To create your own account choose the **Create Account** tab, enter a full name, a user name (3-20 letters, digits, dot, dash or underscore) and a password of at least four characters, confirm it and press **Create Account**.

3. To try the program without an account press **Continue as Guest**.
4. The eye icon in a password field shows or hides the typed password.

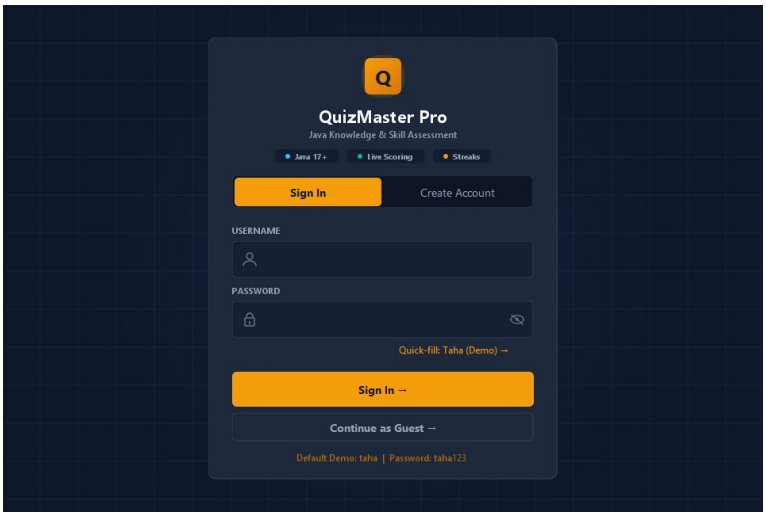


Figure 17: User manual: login screen

M5. Dashboard and Navigation

After signing in the dashboard shows your statistics and recent activity. Use the icons in the left rail to move between screens: Home (dashboard), Quizzes (start a quiz), History, Settings (bottom) and Log Out (bottom). Hover over an icon to see its name. Press **Start New Quiz** to begin.

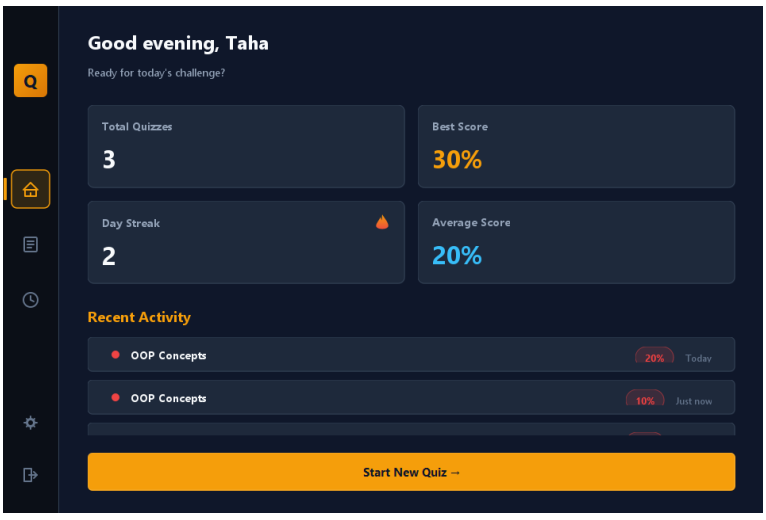


Figure 18: User manual: dashboard

M6. Taking a Quiz

1. Choose a topic by clicking one of the four category cards; the selected card is outlined in amber.
2. Choose the number of questions (5, 10 or 15) and press **Start Quiz**.

3. Read the question. The ring at the top right counts down the time left for the question.
4. Select an answer by clicking an option or pressing **1-4** or **A-D**, then press **Confirm Answer** or **Enter**.
5. The correct option turns green; a wrong choice turns red; an explanation appears below. Press **Next Question** or **Enter** to continue.
6. **Skip Question** moves on without answering. If the timer reaches zero, the selected option is submitted, or the question is skipped if nothing was selected.
7. Use **50:50** to remove two wrong options and **View Hint** to see a hint. Each can be used only once per quiz.
8. Press **Quit** (or any other navigation item) to leave a quiz; the program asks for confirmation because progress is lost.

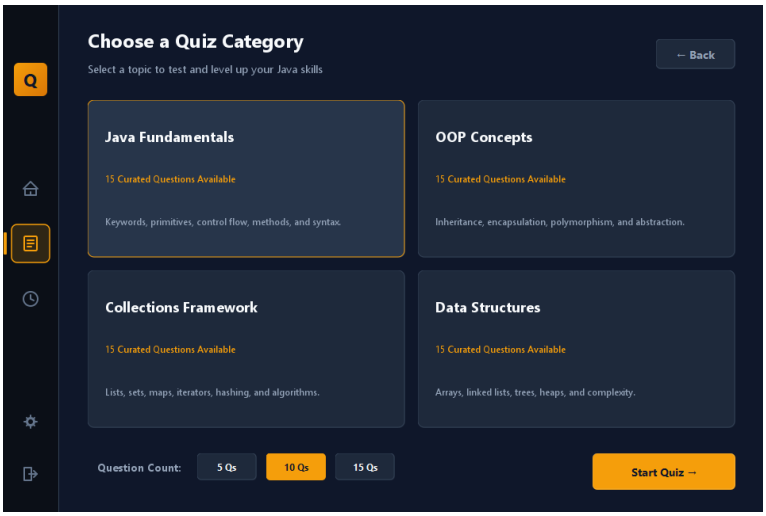


Figure 19: User manual: choosing a category and number of questions

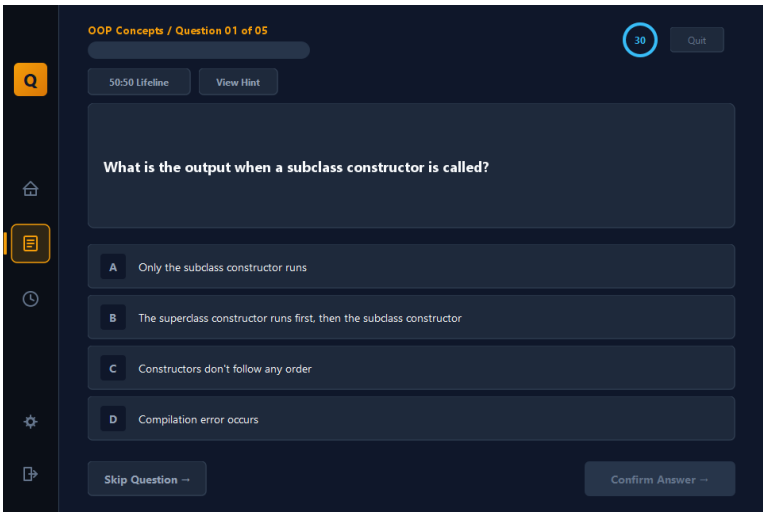


Figure 20: User manual: answering a question

M7. Keyboard Shortcuts

Table 11: Keyboard shortcuts during a quiz

Key	Action
1, 2, 3, 4 or A, B, C, D	Select option 1-4 (A-D)
Enter	Confirm the selected answer; after confirming, go to the next question or to the results
Tab / Shift+Tab	Move the keyboard focus between buttons (a blue ring shows the focused button)
Esc	Close the answer-review dialog

M8. Results and Review

When the last question is finished the result screen shows your score on a ring, the numbers of correct, wrong and skipped questions and your grade (A+ from 95 %, A from 80 %, B from 65 %, C from 50 %, otherwise F). Press **Try Again** to repeat the same category, **New Quiz** to choose another, or **Review Answers** to see every question with your answer, the correct answer and the explanation.

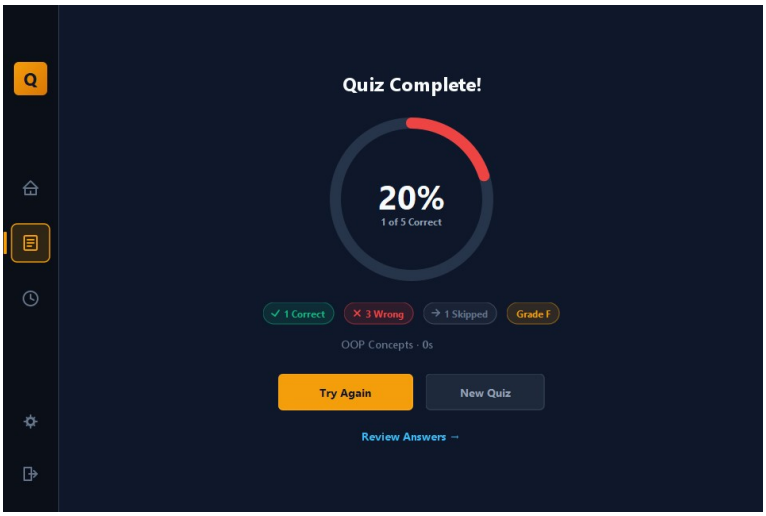


Figure 21: User manual: result screen

M9. History and Settings

The **History** screen lists your most recent attempts (up to 50) with date, duration, score, correct count and grade. The **Settings** screen lets you change your display name, switch sound on or off, set the question timer to 15, 30, 45 or 60 seconds, sign out, or reset all statistics and history (a confirmation is required and the reset cannot be undone).

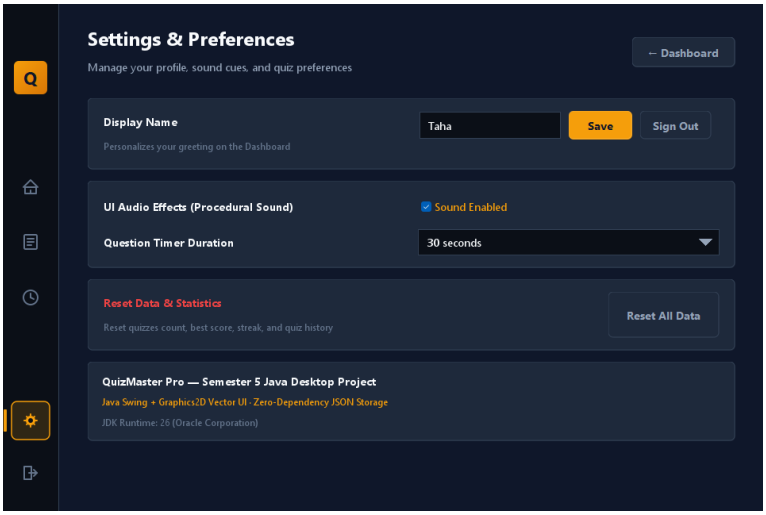


Figure 22: User manual: settings screen

M10. Troubleshooting

Table 12: Common problems and solutions

Problem	Solution
"javac" is not recognised	Install a JDK (not only a JRE) and make sure its bin folder is in the PATH, then open a new terminal.
The window does not start or no questions are shown	Start the program from the project folder so that the data folder is found; do not move or rename data/questions.
Text looks different or fonts are missing	The design uses the Segoe UI font of Windows; other systems use a substitute font.
No sound	Open Settings and tick "Sound Enabled"; check the system volume.
Forgot the password	Create a new account, or delete the files in data/users to reset all accounts (the demonstration account is re-created automatically).

ACHIEVEMENTS

Add your achievements here, for example: presentation of the project in class, certificates, hackathon or competition participation, publication, or recognition by the faculty. If there are none, write "Not applicable".

Project outcomes:

- A complete Java Swing application of 33 classes and about 6,300 lines of code with no third-party dependencies.
- A 60-question Java question bank in four categories.
- 37 automated logic checks and 11 end-to-end checks, all passing.
- A public GitHub repository with build scripts and documentation: github.com/SHZunknown11/quizmaster-pro.